

TOI 19th at NU

Report from Scientific Committee

Contest Report

Incident Day 1

- 08:40 เริ่มทำการสอบ
- มีขอความช่วยเหลือ 8 ครั้ง
 - มีต้องย้ายเครื่อง 3 คน ต่อเวลาให้ 2, 3, 5 นาที
 - ที่เหลือเป็นโปรแกรมค้างจากการรันโค้ดของตัวเอง ไม่ต่อเวลา

Incident Day 2

- 08:25 ตรวจพบความผิดพลาดของ testcase
- 08:35 แก้ไขระบบเรียบร้อยแล้ว
- 08:40 เริ่มทำการสอบ
- มีขอความช่วยเหลือ 3 ครั้ง
 - เครื่องค้าง ต้อง restart, network problem ต่อเวลาให้ 2, 3, 3 นาที

PROBLEM REPORT

การเปลี่ยนแปลงแนวทางข้อสอบ

- เพิ่มเวลาการสอบเป็น 4 ชม.
 - เพื่อให้สามารถออกโจทย์ที่ต้องใช้เวลาทำได้มากขึ้น
- มีข้อที่มีการให้คะแนนแบบ partial scoring
- รับข้อสอบจากศูนย์ต่าง ๆ

แนวทางในการทำข้อสอบ

- ถ้าคิด full solution ไม่ได้ อยากรู้ให้พยายามทำทีละ subtask

Time	Task	Status
03:12:4	d2p1 shopping	▶ Scored (0.0 / 100.0)
02:58:2	d2p1 shopping	▶ Scored (17.0 / 100.0)
02:25:1	d2p1 shopping	▶ Scored (8.0 / 100.0)
02:15:0	d2p1 shopping	▶ Scored (13.0 / 100.0)
02:13:5	d2p1 shopping	▶ Scored (13.0 / 100.0)
02:13:4	d2p1 shopping	▶ Compilation failed

Subtask 3

Subtask 2

Subtask 1

ได้คะแนนรวม 38

Solution Count

Problem	Official Solution	Subtask solution	#Subtask
D1: Merge	2	6	5
D1: Explorer (ปรับปรุงมาจากศูนย์ ม.เชียงใหม่)	5	6	7
D1: Jewelry (ออกโดยศูนย์ ม.บูรพา)	2	7	7
D2: Energy (ออกโดยศูนย์ ม.ธรรมศาสตร์)	3	10	6
D2: Phitsanulok	2	13	9
D2: Range	4	10	7 (with partial)

Merge

- Basic Solution

- Merge 2 รายการเข้าด้วยกัน สร้างรายการ U
- ทดลองนับความถี่ของข้อมูลจนถึง slot ที่ต้องการ
 - สร้าง Cumulative Sum array ขึ้นมา โดยให้ $C[i]$ คือผลรวมของ $u[1] + \dots + u[i]$ แล้วใช้ Binary Search หาช่อง $C[i]$ ที่น้อยที่สุดที่มากกว่าเท่ากับ k

- Official Solution $O(Q \log^2(n))$

- Binary Search สองชั้น
- ชั้นแรกแบ่งเป็นสองคำถาม
 - จากค่า ระยะทางที่กำหนดให้เป็น P นั้น เป็น slot ที่เท่าไรของนาย X (ใช้ binary search)
 - จากค่า ระยะทางที่กำหนดให้เป็น P นั้น เป็น slot ที่เท่าไรของนาย Y (ใช้ binary search)
 - นำคำตอบทั้งสองมาบวกกัน จะสามารถตอบได้ว่า P เป็น slot เท่าไรของข้อมูลที่ผสมแล้ว
- ชั้นสองคือ ลองหาค่า P สำหรับคำถามแรกที่ได้ตอบจากคำถามแรกเป็น slot ที่ต้องการ

Merge Subtask

- (8 คะแนน) small testcase
 - ทำ brute force ตรง ๆ
- (13 คะแนน) x และ y มีในทุกช่อง
 - ทำคล้าย ๆ merge sort ได้
 - วิธี merge จะง่ายขึ้น
- (17 คะแนน) α, β ไม่เปลี่ยน
 - ให้ merge ก่อนแล้วใช้ binary search
- (22 คะแนน) $n = 1, s1 = 1$
 - search บน y แล้วพิจารณาว่า x ตัวนั้นอยู่ก่อนหรือหลัง แล้วปรับคำตอบไป 1 ช่อง
 - ใ้ว่าต้องเขียน binary search ด้วยตัวเอง

Explorer Official Solution $O(n)$

- สร้างต้นไม้ที่แสดงถึงเขาวงกต
- สังเกตว่าวิธีการสำรวจเป็น Depth First Search ดังนั้น หากให้รากของต้นไม้คือ r และ ลูก ๆ ของ r คือ c_1 ถึง c_p เราจะเห็นว่าเราจะสำรวจต้นไม้ย่อยที่มีราก เป็น c_i ให้เสร็จก่อนทั้งต้น ถึงจะเริ่มไปสำรวจต้นไม้ย่อยที่มีรากเป็น c_j และเราสามารถสลับลำดับการสำรวจต้นไม้ย่อยที่ 1 ถึง p อย่างไม่รู้ก็ได้ (แต่ต้องสำรวจต้นไม้ย่อยให้เสร็จทั้งต้นก่อนไปสำรวจต้นไม้ย่อยอื่น)
- หากให้ $\text{count}(x)$ คือการถามว่าจำนวนรูปแบบการสำรวจ ต้นไม้ที่มีรากเป็น p คือเท่าไร จะได้ว่า
$$\text{count}(x) = p! * \text{count}(c_1) * \text{count}(c_2) * \dots * \text{count}(c_p)$$
- สามารถสร้างต้นไม้จากรายการสำรวจของนักสำรวจในอดีตได้ โดยสังเกตว่าการสำรวจต้นไม้ที่มีรากเป็น x ไດ ๆ จะมีผลสำรวจเป็น $[x, a_1, a_2, a_3, a_4, a_5, x, b_1, x, c_1, c_2, c_3, x, d_1, d_2, d_3, x]$ กล่าวคือค่า x จะเป็นตัวแบ่งระหว่างผลการสำรวจของต้นไม้ย่อย

Explorer Subtask

- (3 คะแนน) ต้นไม้ลึก 1
 - ตอบ $(n-1)!$ (Math)
- (11 คะแนน) $N = 4$
 - ให้ทำ case analysis ลองวาดด้วยมือดูว่าต้นไม้เป็นอะไรได้บ้าง (Math)
- (10 คะแนน) Complete D-Ary Tree
 - ถ้ารู้ความลึกและค่า D จะสามารถหาคำตอบจากสูตรคำนวณได้เลย
 - ต้องการให้รู้ว่าสามารถดูรายการสำรวจแล้ววิเคราะห์ความลึกกับค่า D ได้โดยพิจารณาตัวซ้ำ และ จำนวนครั้งที่ x ปรากฏในรายการ
- (9 คะแนน) ต้นไม้ลึก 2
 - ทำเหมือน subtask 1 เป็นสองครั้ง ไม่ต้อง recursive
- (11 คะแนน) Binary Tree ขนาดเล็ก ๆ
 - recursive ได้
- (16 คะแนน) Binary Tree ขนาดใหญ่ ๆ

Jewelry

- Official Solution $O(N)$
 - Dynamic Programming คล้าย ๆ Kadane's Algorithm
 - ตำแหน่งของ T จะมีความสัมพันธ์กับผลลัพธ์สุดท้าย
- กรณีที่ไม่มีค่า F ในสาย (case TTTT...TTT)
 - ให้ $R(N)$ คือคำตอบเมื่อพิจารณาถึงตัวที่ N
 - $R(N)$ ได้จากสมการ
$$R(N) = \sum_{i=1}^N (N - i)i$$
 - ได้เป็น $R(i) = R(i-1) + i$ โดย $i = 1 \sim N$
- คำตอบคือ sum ของ $R(1) \dots R(N)$

Jewelry

- CASE: TTTT...TTTTT

จะได้ว่า $R(i) = R(i-1) + i$ โดย $i = 1 \sim N$

	T	T	T	T	T	T
	1	2	3	4	5	6
1	1	2	3	4	5	6
2		1	2	3	4	5
3			1	2	3	4
4				1	2	3
5					1	2
6						1

ตำแหน่ง
จบ

ตำแหน่ง
เริ่ม

Jewelry

- CASE: FF..FFTT...TTT

	F	F	T	T	T	T
	1	2	3	4	5	6
1	0	0	1	2	3	4
2		0	1	2	3	4
3			1	2	3	4
4				1	2	3
5					1	2
6						1

จะได้ว่า $R(i) =$

- $R(i-1)$
- $R(i-1) + i$

เมื่อข้อมูลตัวที่ i เป็น F
เมื่อข้อมูลตัวที่ i เป็น T

Jewelry

- CASE: TT...TTFFF...FFFTTT...TT

	T	F	F	T	T	T
	1	2	3	4	5	6
1	1	1	1	1	2	3
2		0	0	1	2	3
3			0	1	2	3
4				1	2	3
5					1	2
6						1

มีความสัมพันธ์ของตำแหน่ง T ในกลุ่มก่อน
หน้ามาเกี่ยวข้อง

จะได้ว่า $R(i) =$

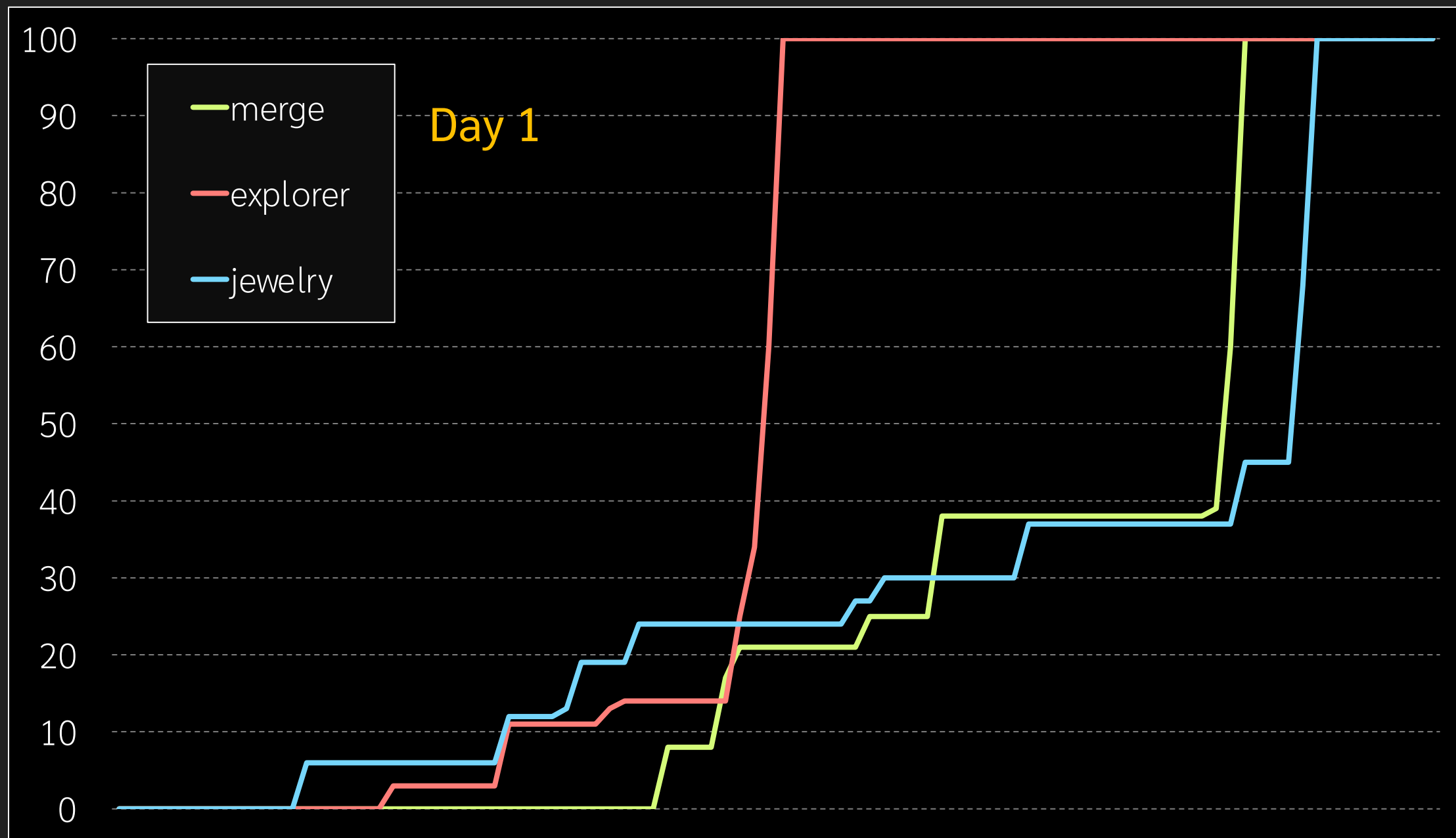
- $R(i-1)$ เมื่อข้อมูลตัวที่ i เป็น F
- $R(i-1) + i - R(x)$ เมื่อข้อมูลตัวที่ i เป็น T

x เป็นตำแหน่งที่ T ตัวแรกสุดที่ทำให้ค่า T
ที่ติดกันมากที่สุดเปลี่ยนแปลง

x คำนวณโดยใช้ DP อีกตัว

Jewelry Subtask

- (6 คะแนน) : $N \leq 300$ (N^3 ทำ simple for loop นับตรง ๆ)
- (18 คะแนน) : $N \leq 5000$
 - $O(N^2)$ ทำ for loop แต่นับรวมแบบเร็ว ๆ เป็น base ของการคิด Dynamic Programming
- (6 คะแนน) : FFFF...FFFTTTT...TTTT
- (7 คะแนน) : FFFF...FFFTTTT...TTTTFFFFF...FFFF
- (8 คะแนน) : TTTT...TTTTFFFFF...FFFTTTT...TTTT
 - Base ของการคิด Dynamic Programming
- (23 คะแนน) : ไม่มีตัวซ้ำกันเกิน 100 ตัวติดกัน
 - $O(N*100)$ ทำ คล้าย ๆ Full Solution แต่ตอนหา x (เป็นตำแหน่งที่ T ตัวแรกสุดที่ทำให้ค่า T ที่ติดกันมากที่สุดเปลี่ยนแปลง) ไม่ต้องใช้ dynamic มาจำ ใช้ for loop ได้เลย



Score Summary – Day 1

โจทย์	คนเต็ม	คนได้คะแนน	คะแนนรวม	จำนวน sub
Merge	14	54	2,584	581
Explorer	46	73	4,945	370
Jewelry	9	79	2,638	760

มีคนไม่ได้คะแนน 4 คน

Energy

- Official Solution $O(kN^3)$
- Classical Dynamic Programming
 - ขยายมาจาก Matrix Chain Multiplication
- ให้ $C(l,r,M)$ เป็นจำนวนรูปแบบการแบ่งเซลล์พลังงานสำหรับปมในชั้นที่ M โดยใช้เฉพาะเซลล์หมายเลข l ถึง r จะได้ว่า $C(l,r,M)$ มีค่าเป็น
 - 0 หาก $(l < r)$
 - 1 หาก $(d = k)$ (ชั้นสุดท้าย แบ่งได้วิธีเดียวคือใช้ทุกเซลล์)
 - ผลรวมของ $C(l,x,d+1) * C(x+1,r,d+1)$ สำหรับ x ตั้งแต่ l ถึง $r-1$ และ $\text{sum}(e[l..x]) - \text{sum}(e[x+1..r]) \leq D$

Energy Subtask

- (2 คะแนน) : $K = 2$
 - for loop 1 ชั้น เพื่อแบ่งเป็น 2 ส่วน
- (8 คะแนน) : $K = 3, N \leq 50$
 - For loop 3 ชั้นเพื่อแบ่งเป็น 4 ส่วน
- (6 คะแนน) : $K = 3$
 - ใ้ว่าต้องใช้ Quick Sum ช่วย
- (17 คะแนน) : มีเซลล์เกินมา 3
 - สร้างวิธีการแบ่งทุกรูปแบบที่เป็นไปได้ โดยใช้ start and bar
 - ไม่ต้องทำ Dynamic Programming
- (28 คะแนน): $N \leq 50$
 - ทำได้โดยไม่ต้อง Dynamic Programming แต่แก้ Recurrence Relation แบบ recursive ตรง ๆ

Phitsanulok Base Solution $O(N^2)$

- สร้างกราฟขึ้นมา ให้ ปมคือผลไม้แต่ละผล และมีเส้นเชื่อมจาก $a \rightarrow b$ ก็ต่อเมื่อเราสามารถกินผลไม้ b ต่อจากผลไม้ a ได้ (ก็คือผลไม้ b รักษาพิษจากผลไม้ a)
 - น้ำหนักของเส้นเชื่อมคือ น้ำหนักของผลไม้ที่ต้องกิน
 - เวลาในการสร้างกราฟคือ $O(N^2)$
- นู๊ก็ จะเลือกผลไม้เริ่มต้น น้อนอ๋ม จะต้องหา shortest path จากปมเริ่มต้นนั้นไปยังปมที่ไม่มีพิษที่ใกล้ที่สุด (single source shortest path)
 - ลองหา shortest path จากทุก ๆ ปมที่เป็นไปได้ (ต้องทำ Shortest Path n ครั้ง)
- ทำให้ดีขึ้นโดย กลับทิศของกราฟ (คือ มีเส้นจาก $a \leftarrow b$ แทนเส้นจาก $a \rightarrow b$) แล้วหา Shortest path จาก ปมที่ไม่มีพิษทั้งหมดแทน
 - ใช้ shortest path รอบเดียว
 - Shortest Path Dijkstra ใช้เวลาเป็น $E \log(N)$ เมื่อ E คือจำนวนเส้นเชื่อม แต่โจทย์นี้มีเส้นเชื่อมจากการรักษาพิษไม่เยอะ

Phitsanulok Subtask

- (9 คะแนน) : $n \leq 8$ (ลองวิธีการกินทุกรูปแบบ)
- (4 คะแนน) : กราฟเป็นเส้นตรงหลาย ๆ เส้น
 - ไปเรื่องการหาต้นทางที่ไกลที่สุด
- (6 คะแนน) : กราฟเส้นตรง 1 ทางเชื่อม
 - ไปให้เห็นความเป็นกราฟ
- (9 คะแนน) : $N \leq 5000$ น้ำหนักเป็น 1
 - หา shortest path ด้วย Breadth First Search
- (8 คะแนน) : $N \leq 700$
 - หา shortest path ด้วย Floyd Warshall $O(N^3)$
- (7 คะแนน) : $N \leq 1500$
 - หา shortest path ด้วย Dijkstra N รอบ $O(N E \log(n))$
- (22 คะแนน) : $N \leq 5000$
 - ทำ reverse graph หา shortest path ด้วย Dijkstra 1 รอบ $O(E \log(n))$ แต่เสียแรงสร้างกราฟ $O(N^2)$

2 subtask สุดท้ายมี N เกิน 50,000 ทำ $O(N^2)$ เพื่อสร้างกราฟไม่ไหว
ต้องหาทางทำกราฟ
แบบอื่น

Phitsanulok Official Solution $O(N \log(2^S))$

- ให้สังเกตว่า น๊อนอัม จะติดพิชได้ 2^S แบบ และมีผลไม้ถอนพิชที่เป็นไปได้อยู่ 2^S แบบเช่นกัน
 - หาเลขฐาน 2 แสดงการติดพิช และ การถอนพิช
 - เช่น ผลไม้ 0 1 1 -1 0 -1 ทำให้ถอนพิชแบบ $(001100)_2$ ติดพิชแบบ $(000101)_2$
- สร้างกราฟใหม่ ให้ปมแต่ละปมแทนสถานะติดพิชและความสามารถในการถอนพิช และมีเส้นเชื่อมจากปม $a \rightarrow b$ ก็ต่อเมื่อมีผลไม้ที่ถอนพิช a แต่ทำให้ติดพิช b
 - เช่น ผลไม้ 0 1 1 -1 0 1 ข้างต้น จะทำให้มีเส้นเชื่อมจากปม $(001100)_2$ ไปยังปม $(000101)_2$
 - ใช้วิธีเดียวกับ base solution คือหา shortest path ไปยังปม (0000000) คือปมไม่ติดพิชเลย
 - ทำ
 - ทำกราฟย้อนกลับแทนเพื่อใช้ Single Source Shortest Path รอบเดียว
 - มี 2^S ปม และมี N เส้นเชื่อม เวลาที่ใช้คือ $N \lg(2^S)$

Phitsanulok Official Solution Example

- ต้องมีเส้นเชื่อม “ช่วยเหลือ” เพื่อ
บอกว่า ถอนพิษมากกว่าพิษได้ เช่น
ติดพิษ -1 0 0 สามารถช่วยได้ด้วย

1 0 0

1 1 0

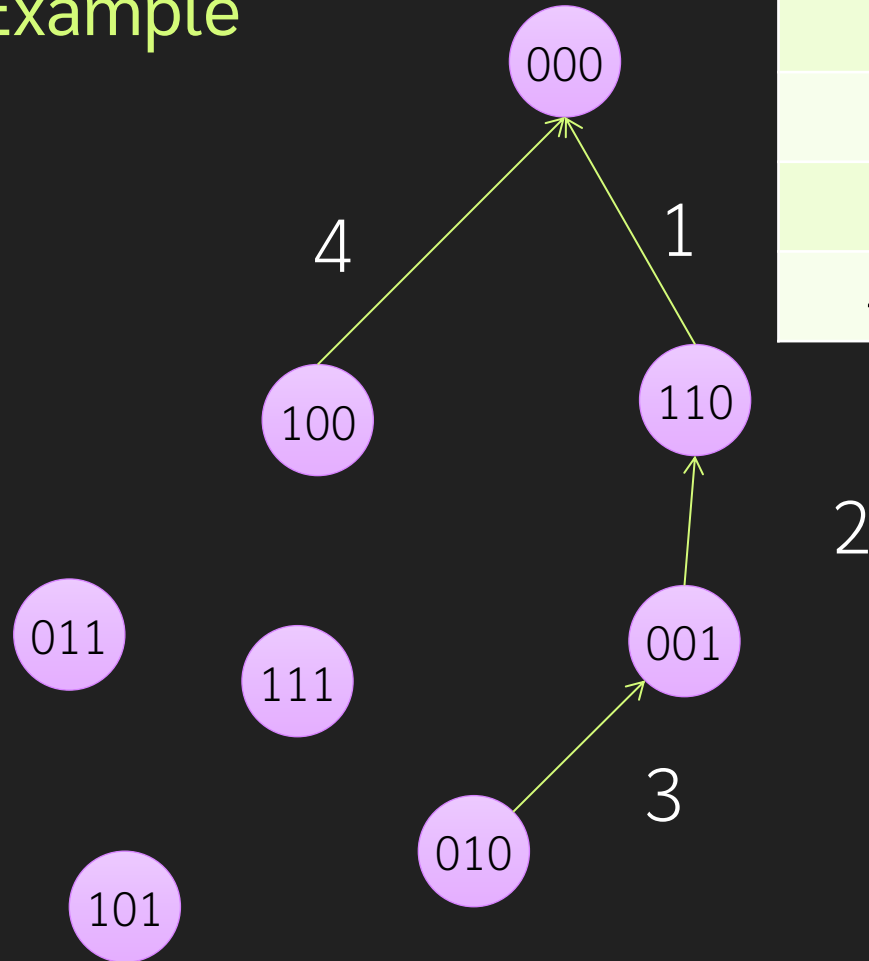
1 0 1

1 1 1 เป็นต้น โดยมีเส้นเชื่อม
น้ำหนัก 0 เชื่อมจาก 1 0 0 ไป

1 1 0

1 0 1

1 1 1 ด้วย

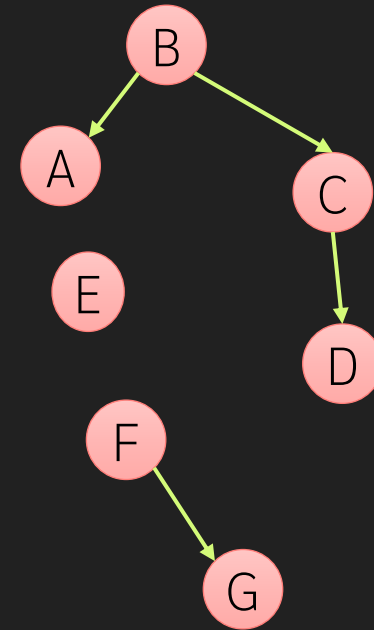
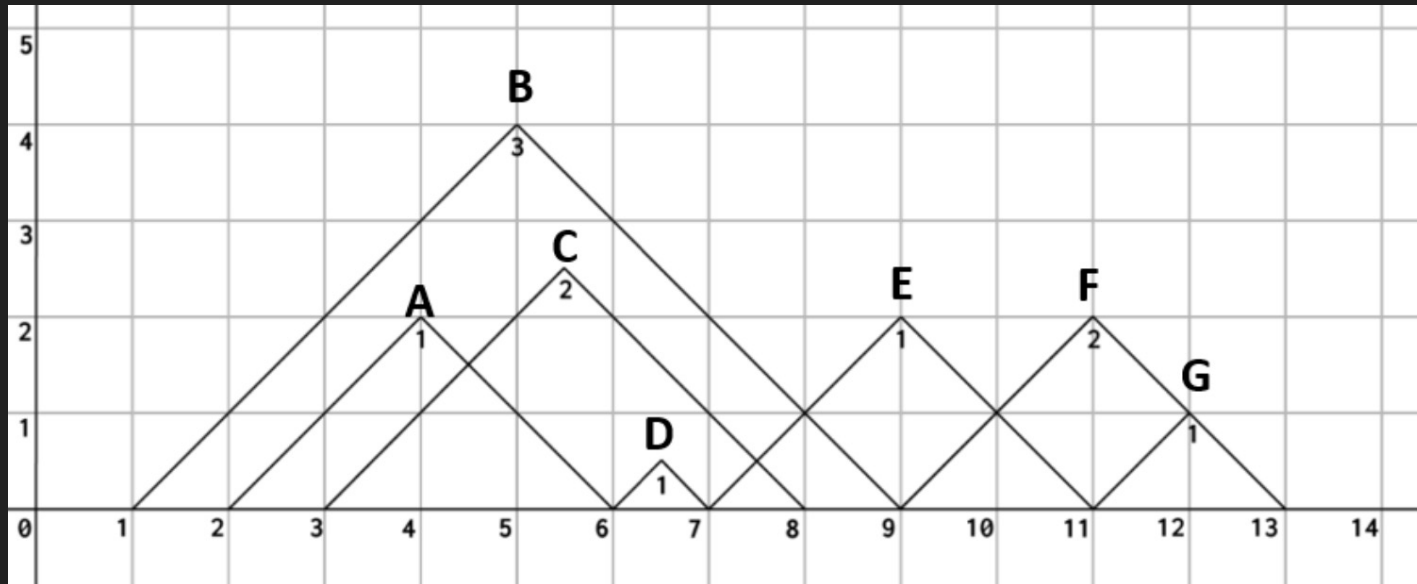


w	1	2	3
1	1	1	0
2	-1	-1	1
3	0	1	-1
4	1	0	0

แปลงกราฟย้อนกลับแล้วทำ
Dijkstra ตามปกติ

Range Base Solution $O(N^2)$

- สร้างกราฟ โดยปมคือภูเขา และมีเส้นเชื่อมจากปม $a \rightarrow b$ ก็ต่อเมื่อ a เป็นพี่ของ b ได้เป็น Directed Acyclic Graph
- หา Longest Path จากแต่ละปมใน DAG ใช้ DFS ได้เลย
- คำตอบคือ Longest Path ที่ยาวที่สุดจากแต่ละปม



Range Partial Solution $O(N \lg N)$

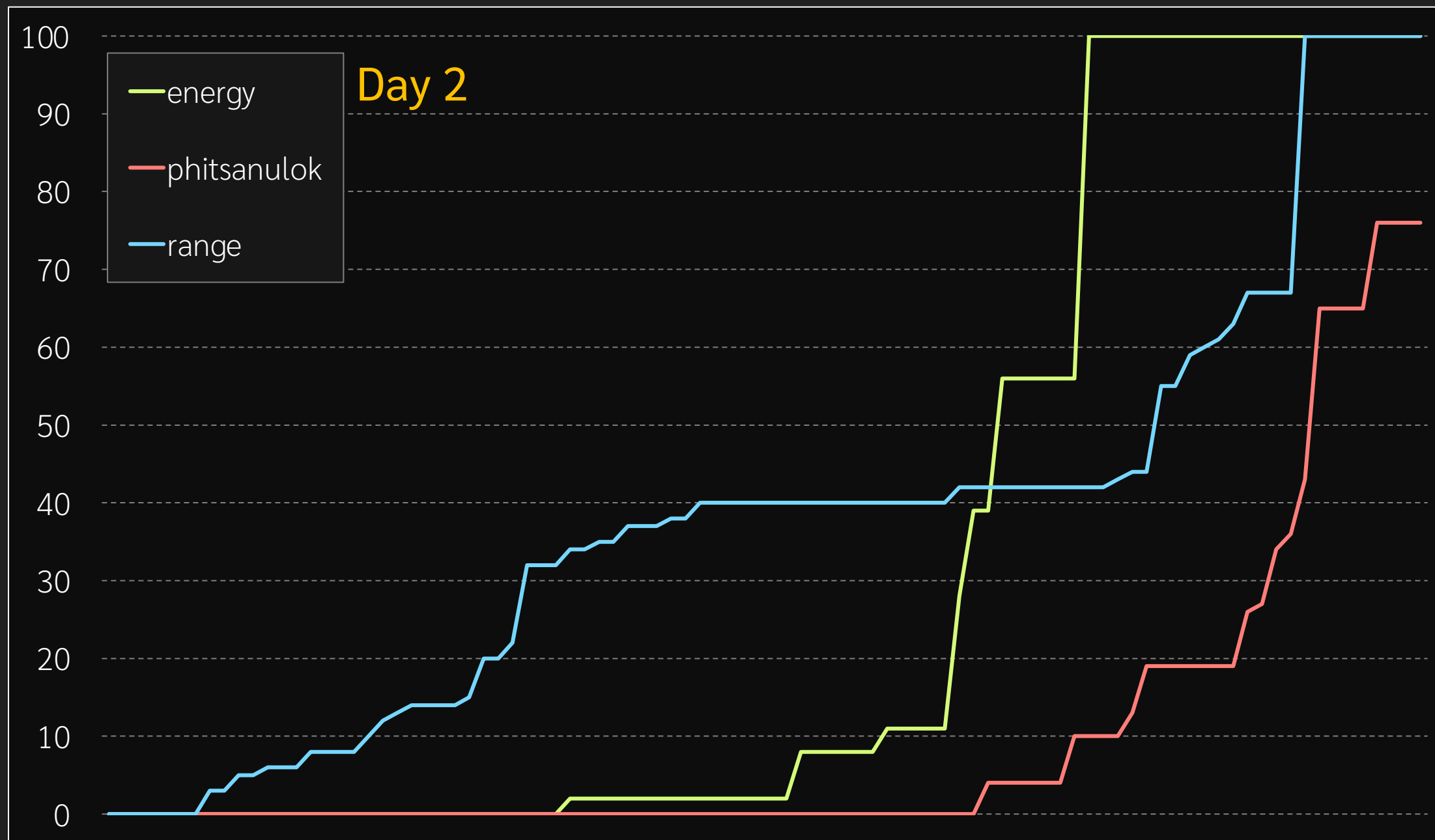
- ภูเขา a, b, c, d เป็นพี่น้องกันไปเรื่อย ๆ แปลว่า $L_a \leq L_b \leq L_c \leq L_d \leq R_d \leq R_c \leq R_b \leq R_a$
- สร้างอาร์เรย์ของค่า L แต่เรียงอาร์เรย์นี้ตามค่า R จากมากไปน้อย
 - เช่น มีภูเขาเป็น [1, 40], [2, 30], [3, 35], [4, 25] ให้สร้างอาร์เรย์เป็น [1, 3, 2, 4]
 - ให้ดู increasing sequence ในนี้ จะสัมพันธ์กับ decreasing sequence ของค่า R
 - เช่น [1, 3, 4] จะหมายถึง [1, 40], [3, 35], [4, 25] จะเห็นว่าค่า L จะเรียงจากน้อยไปมาก และค่า R เรียงจากมากไปน้อย ซึ่งเป็นลำดับของภูเขาที่เป็นพี่น้องกันพอดี
- คำตอบคือหาความยาวของ Longest Non-Decreasing Subsequence ของอาร์เรย์ดังกล่าว
- ได้คำตอบบรรทัดแรก 40%

Range Full Solution $O(N \lg N)$

- Sort ภูเขาตามค่า R จากน้อยไปมาก โดยเมื่อพิจารณาถึงภูเขาลูกที่ i จากที่เรียงแล้วนี้ เราจะรับประกันได้ว่าภูเขาลูกที่ i จะเป็นพีได้เฉพาะกับภูเขาลูกที่ 1 ถึง $i-1$ เท่านั้น
- เมื่อเรียงตามค่า R จากน้อยไปมากแล้ว ก็ใช้ลำดับดังกล่าวสร้างอาร์เรย์ของค่า L แล้วหา Longest Non-Increasing Subsequence (LNIS) แทน แล้วถามว่า เมื่อพิจารณาภูเขาลูกที่ i นั้น มี LNIS ที่จบด้วย i เป็นความยาวเท่าไร
 - ต้องสร้างตัวแปรมาจำว่าภูเขาลูกนั้น ๆ เป็นลูกที่เท่าไรในข้อมูลนำเข้าเพื่อนำไปตอบในบรรทัดที่ 2
 - [4,25] เลือก 4 ตอบ 1
 - [4,25], [2, 30] เลือก 4, 2 ตอบ 2
 - [4,25], [2, 30], [3, 35] เลือก 4, 3 ตอบ 2
 - [4,25], [2, 30], [3, 35], [1, 40] เลือก 4, 2, 1 ตอบ 3

Range subtask

- (5 คะแนน) $L = 1$ พิจารณาเฉพาะ R เท่านั้น
- (4 คะแนน) : ภูเขาเรียงตัวกันเป็นพี่น้องอยู่แล้ว
 - ใ้ว่าต้องแปลงเป็นกราฟแล้วหา path ให้เจอ
- (6 คะแนน) : L เรียงจากน้อยไปมาก, R เรียงจากน้อยไปมากพร้อม ๆ กัน
 - ใ้ว่าต้องการ shortest path บน unweighted graph (ใช้ BFS ได้)
- (9 คะแนน) : $N = 5$
 - ทำ permutation
- (12 คะแนน) $N = 2000$
 - LNDS แบบ $O(N^2)$
- (11 คะแนน) : ไม่เกิน 50 ชั้น
 - Binary Search สร้าง ต้นไม้



Score Summary – Day 2

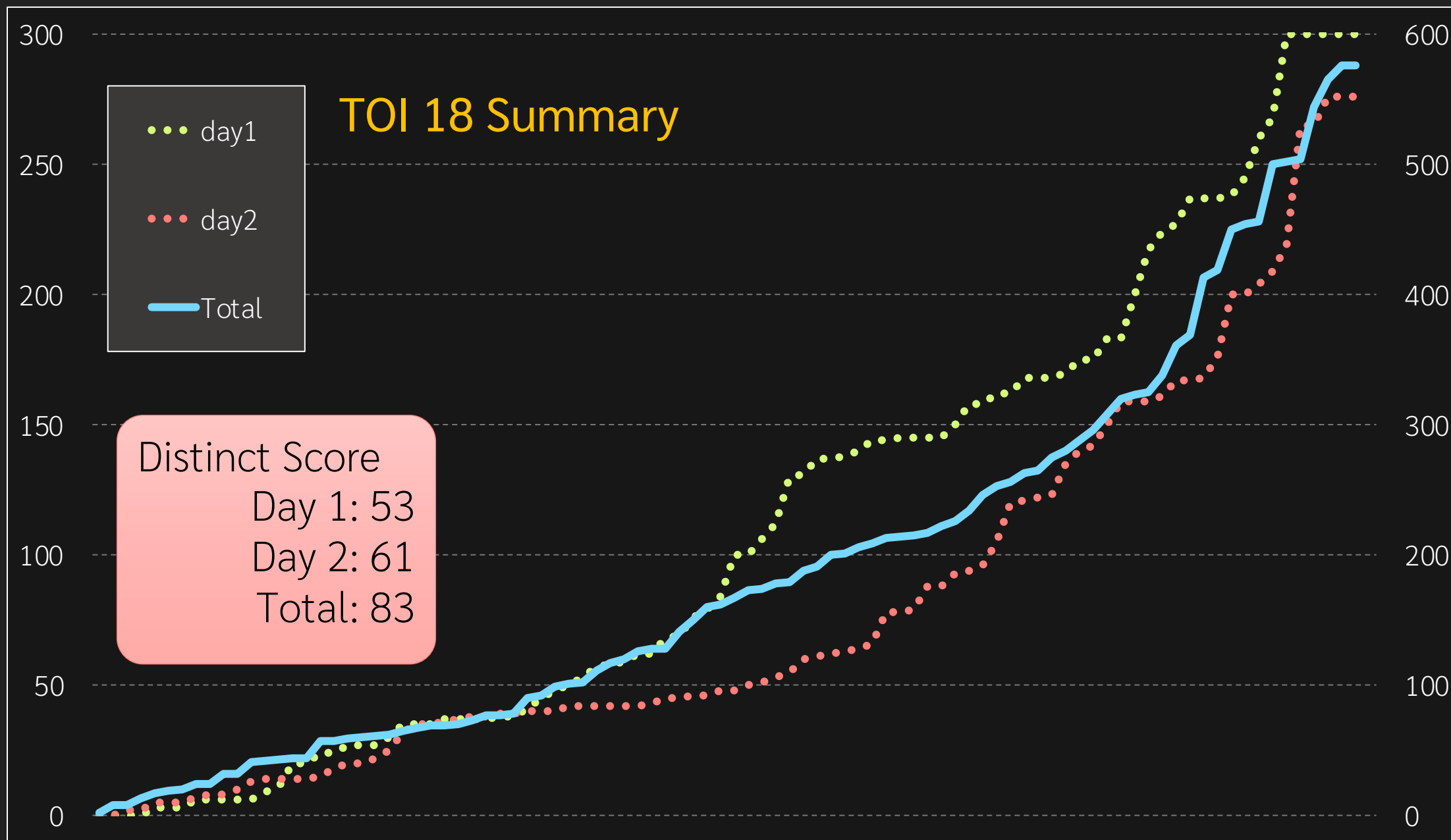
โจทย์	คนเต็ม	คนได้คะแนน	คะแนนรวม	จำนวน sub
Energy	24	60	2,977	586
Phitsanulok	0	31	940	453
Range	9	85	3,489	1,006

มีคนไม่ได้คะแนน 2 คน

TOI 18 Summary



Distinct Score
Day 1: 53
Day 2: 61
Total: 83



Medal Allocation

Medal Allocation

- S5.10 ก่อนพิธีมอบรางวัล TOI นั้น SC ต้องยื่นเรื่องต่อ GA เกี่ยวกับการมอบเหรียญรางวัลให้ผู้เข้าแข่งขันตามคะแนนที่ผู้เข้าแข่งขันได้รับ GA ต้องยืนยันคะแนนและรางวัลของผู้เข้าแข่งขัน ผู้เข้าแข่งขันไม่เกินครึ่งหนึ่งจะได้รับเหรียญรางวัลตามหลักดังนี้
 - ผู้เข้าแข่งขันประมาณหนึ่งในสิบสองของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญทอง
 - ผู้เข้าแข่งขันประมาณหนึ่งในหกของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญเงิน
 - ผู้เข้าแข่งขันประมาณหนึ่งในสี่ของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญทองแดง
- GA ต้องยืนยันข้อเสนอในการจัดสรรเหรียญรางวัลก่อนให้คณะผู้แทนทั้งหมดรับทราบผลคะแนน

Competitor

- รายชื่อทั้งหมดมีนักเรียน 92 คน
 - Fossil 2 คน
- จำนวนคนที่ใช้คิดในการตัดแบ่งเหรียญรางวัล $= 92 - 2 = 90$ คน

Medal Allocation Proposal (IOI-based)

- การคำนวณเหรียญจะใช้จำนวนคน 90 คน
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองคือคะแนนสูงสุดที่ทำให้มีอย่างน้อย $90/12 = 7.5 \sim 8$ คนได้เหรียญทอง
- เกณฑ์คะแนนขั้นต่ำของเหรียญเงินคือคะแนนสูงสุดที่ทำให้มีอย่างน้อย $90/6 = 15$ คนได้เหรียญเงิน
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองแดงคือคะแนนต่ำสุดที่ทำให้ไม่เกิน $90/2 = 45$ คนได้เหรียญใดเหรียญหนึ่ง แปลว่าต้องมีทองแดง 23 คน (มากกว่าปีที่แล้ว 2 คน)
 - เนื่องจาก $7+15+23 = 45$
 - เหรียญทองสุดท้าย – เหรียญเงินแรก = 5 คะแนน
 - เหรียญเงินสุดท้าย – เหรียญทองแดงแรก = 1 คะแนน
 - เหรียญทองสุดท้าย – เหรียญทองแดงแรก = 123 คะแนน

Medal Allocation Proposal Option

(45 คน)	จำนวน	คะแนนต่างจาก rank ถัดไป
ทอง	8	2
เงิน	15	2
ทองแดง	22	2

(45 คน)	จำนวน	คะแนนต่างจาก rank ถัดไป
ทอง	7	44
เงิน	15	10
ทองแดง	23	2

ที่ประชุมมีมติ
เลือกแบบนี้
(10/15 เสียง)

(44 คน)	จำนวน	คะแนนต่างจาก rank ถัดไป
ทอง	7	44
เงิน	15	10
ทองแดง	22	5

(46 คน)	จำนวน	คะแนนต่างจาก rank ถัดไป
ทอง	8	2
เงิน	15	2
ทองแดง	23	10

(46 คน)	จำนวน	คะแนนต่างจาก rank ถัดไป
ทอง	7	44
เงิน	15	10
ทองแดง	24	10

Issue and Improvement

From TOI18

- (คำแนะนำจาก GA) ปีหน้าลองพยายามรันซ้ำ 1 ครั้งพอ
 - ทำได้หรือไม่ให้เจ้าภาพเป็นผู้กำหนด

DONE IN TOI 19

- (คำแนะนำจาก GA) ปีหน้าลองพยายามเขียนแบบส่ง function อย่างเดียวพอ
 - ทำได้หรือไม่ให้เจ้าภาพเป็นผู้กำหนด

TRY AGAIN
NEXT YEAR

Proposal for TOI 19

- แนวทางเพิ่มเติมของโจทย์
 - เขียนเฉพาะฟังก์ชัน
 - โจทย์ Interactive
- อยากให้เลิกใช้ Code::Blocks
 - หรือไม่ก็ให้นักเรียน kill process เองเป็น
 - จะไม่ต่อเวลาให้จากการค้างของ Code::Blocks
- อยากให้ศูนย์เปลี่ยนไปใช้ CMS
 - ทึมระบบจาก มช. พร้อมให้คำปรึกษา
 - มีคู่มือให้พร้อมแล้ว

การเขียนโปรแกรมเฉพาะฟังก์ชัน

- เป็นแนวทางที่ IOI ใช้
- ผู้เข้าแข่งขันเขียนเฉพาะฟังก์ชันที่จำเป็นต่อการตอบคำถาม
 - ไม่ต้องเขียนการรับค่าอินพุตจาก keyboard หลีกเลียงปัญหาการต้องเขียน IO เช่น `ios_base::sync_with_stdio`
 - ไม่ต้องเขียนการส่งค่าเอาพุตออกหน้าจอ (หลีกเลียงปัญหา `endl`, “\n”)
- ข้อเสีย
 - จำเป็นต้องใช้ไฟล์ .h ในการเขียนโปรแกรม
 - นักเรียนต้อง compile file หลาย ๆ ไฟล์เป็น
 - คำอธิบายในโจทย์อาจจะยาวขึ้น
- ข้อดี
 - ทำให้สร้างโจทย์แบบ interactive ได้

โจทย์ Interactive

- แทนที่โจทย์จะเป็นการรับ input ครั้งเดียว แล้วตอบผลครั้งเดียว จะเป็นการเรียกฟังก์ชันที่ grader เตรียมไว้
- เป็นโจทย์ที่โผล่มาเรื่อย ๆ ใน IOI
- ข้อเสีย
 - ค่อนข้างซับซ้อนในการเริ่มทำ

ตัวอย่างโจทย์

- มีอาร์เรย์ $A[1..n]$ (เราไม่ได้รับข้อมูลค่าของ $A[i]$ กล่าวคือ เราไม่ได้ A มาเป็น input ของโจทย์ เราทราบเพียงค่า n ว่ามี $A[i]$ กี่ช่อง)
 - $A[i]$ แต่ละช่องไม่ซ้ำกัน
- มีฟังก์ชัน `bool check(int p, int q)` ให้เรียกใช้
 - `check(p,q)` จะคืนค่า `true` ก็ต่อเมื่อ $A[p] > A[q]$
 - เราสามารถเรียก `check` ได้หลาย ๆ ครั้ง
- โปรแกรมจะต้องหาค่า u และ v ที่ทำให้ $A[u]$ มากที่สุด และ $A[v]$ น้อยที่สุด

ตัวอย่าง

- คะแนน ที่ได้แปรผกผันกับ “จำนวนครั้ง” ที่เรียก check
- เรียกเยอะ ๆ ได้คะแนนน้อย
- เรียกน้อย ๆ ได้คะแนนเยอะ

```
pair<int,int> solve(int n) {  
    int p,q;  
    for (int i = 0;i < n;i++) {  
        bool is_max = true;  
        bool is_min = true;  
        for (int j = i+1;j < n;j++) {  
            if (check(i,j) == false) {  
                is_max = false;  
            } else {  
                is_min = false;  
            }  
        }  
        if (is_max) { p = i; }  
        if (is_min) { q = i; }  
    }  
  
    return make_pair(p,q);  
}
```